Architecting a Web-Based Matrix Routing Interface for Blackmagic ATEM Video Switchers

The evolution of live video production has been defined by a relentless increase in signal density. While entry-level broadcast hardware, such as the Blackmagic Design ATEM Mini series, manages a relatively narrow routing scope of four to eight inputs directed toward a single program or auxiliary output [1], enterprise-grade hardware operates on a vastly different scale. The flagship ATEM Constellation 8K features an unprecedented 40 independent 12G-SDI inputs and 24 customizable 12G-SDI auxiliary outputs.[2] Operating a routing matrix of this magnitude using the traditional ATEM Software Control application introduces severe operational friction. The traditional software relies on a destination-first dropdown paradigm: an operator must first select an auxiliary output from a menu, then open a secondary dropdown or palette to assign the desired source.[4] In high-stakes, live-broadcast environments, the cognitive load and physical latency required to execute multiple simultaneous routing changes via dropdown menus are operationally unacceptable.

The proposed solution necessitates the development of custom, web-based middleware that entirely reimagines the routing user interface. The conceptual target is the UI paradigm established by Audinate's Dante Controller, which revolutionized networked audio by replacing linear drop-downs with a two-dimensional XY grid matrix.[6] By representing video inputs as horizontal rows and outputs as vertical columns, a technical director can execute a routing change with a single interaction at the appropriate grid intersection.[8] Furthermore, to seamlessly integrate this capability into tactile broadcast workflows, the software must function as a bridge to Bitfocus Companion, enabling complex, pre-programmed routing presets to be triggered instantly from physical control surfaces such as the Elgato Stream Deck.[9]

This comprehensive technical feasibility study investigates the architectural requirements, protocol limitations, state management strategies, and deployment roadmaps necessary to engineer this custom ATEM matrix routing middleware.

Part 1: Hardware and Network Infrastructure

The physical deployment topology is the foundation of the matrix routing system. A highly available, low-latency control network is paramount to ensuring that software-issued commands execute with the same reliability as a physical button press on an ATEM Advanced Panel.

1.1 Target Device Capabilities and Limitations

The architectural approach must account for the vast functional disparities across the Blackmagic ATEM product line. The ATEM Mini, ATEM Mini Pro, and ATEM Mini Extreme lines are highly integrated prosumer devices.[1] They provide a single HDMI auxiliary output (or dual outputs on the Extreme models) and rely on USB-C or internal hardware for encoding.[1] Routing on an ATEM Mini is generally restricted to selecting which of the HDMI inputs, media players, or program/preview buses feeds the singular HDMI output.

Conversely, the ATEM Constellation HD, 4K, and 8K models function as true enterprise routing cores.[2] The 4 M/E Constellation 8K possesses 40 inputs and 24 completely independent auxiliary outputs, all of which support full standards conversion.[3] Because the Constellation lacks a built-in hardware control panel on its chassis (relying only on a small LCD and basic setup buttons), external network control is an absolute imperative for operation.[11] The middleware must dynamically interrogate the connected ATEM model upon initialization to determine the maximum limits of its routing matrix, gracefully scaling the frontend UI to accommodate a 4x1 grid for a Mini or a 40x24 grid for a Constellation 8K.

1.2 Overcoming Concurrent Network Connection Limits

A highly documented architectural vulnerability of Blackmagic ATEM hardware is the strict, undocumented cap on concurrent network control connections. While Blackmagic Design rarely specifies the exact networking thresholds in official literature [12], extensive field testing and community documentation reveal that 1 M/E and 2 M/E ATEM models generally restrict simultaneous network connections to between 5 and 7 clients.[13] Although modern Constellation models possess slightly higher connection limits, they remain highly susceptible to connection pool exhaustion.[15]

This limitation becomes critical in modern broadcast environments. A typical studio may utilize a hardware ATEM Advanced Panel (1 connection), a primary ATEM Software Control instance (1 connection), an iPad running MixEffect (1 connection), a wireless tally light system (1 connection per tally or via a hub), and multiple instances of Bitfocus Companion (multiple connections).[15] Furthermore, ATEM switchers handle TCP/UDP timeout procedures poorly; if a client, such as an iPad or a wireless tally, momentarily drops its Wi-Fi connection and reconnects, the ATEM frequently allocates a new connection slot to the returning client while keeping the previous "dead" connection active until a lengthy timeout period expires.[14] This cascading failure rapid exhausts the connection pool, rendering the switcher entirely unresponsive to new control commands until the chassis is physically rebooted.[15]

The proposed middleware solves this fundamental vulnerability by implementing a Proxy Design Pattern.[17] Rather than allowing various web clients, stream decks, and applications to connect directly to the ATEM, the custom Node.js server acts as the sole, highly persistent,

wired client interfacing with the hardware.[19]

Table 1: Network Connection Topography Comparison

Architecture Type	Client Connections	ATEM Connections Consumed	Vulnerability to Exhaustion
Direct Distributed Control	3x Web UIs, 2x Stream Decks	5 (High risk)	Critical. Wi-Fi dropouts create zombie connections, exhausting the ATEM pool.[14]
Node.js Proxy Middleware	Unlimited Web UIs, Unlimited Stream Decks	1 (Guaranteed Singleton)	None. Node.js manages TCP client pooling; ATEM sees only one persistent UDP client.[20]

1.3 Host Machine Processing Requirements

To serve as the reliable core of this proxy architecture, the middleware requires a dedicated, always-on host machine deployed on the same local area network (LAN) subnet as the ATEM and the Bitfocus Companion system.[22] Due to their compact form factor, low power draw, and high deployment flexibility, single-board computers like the Raspberry Pi or micro-servers like the Intel NUC are the industry standard for broadcast middleware.

When evaluating the Raspberry Pi hardware stack, it is crucial to recognize the computational overhead of the Node.js event loop when parsing high-frequency UDP traffic. Historical data indicates that poorly optimized Node.js proxy servers handling state management can suffer from memory leaks and CPU throttling on older ARM architectures, such as the Raspberry Pi 3 or 4.[23]

The Raspberry Pi 5 is the recommended baseline target for this deployment. Benchmark testing demonstrates that the Raspberry Pi 5 provides a 2.0x to 2.5x increase in CPU performance over the Raspberry Pi 4.[26] This enhanced processing bandwidth, combined with vastly superior memory architecture, ensures that the Node.js event loop is not blocked during intensive transition periods—such as when a technical director triggers a macro that alters 20 routing crosspoints simultaneously, flooding the server with sequential state change notifications. A hardwired gigabit Ethernet connection is mandatory; the host machine must

never rely on Wi-Fi to communicate with the ATEM, ensuring the UDP proxy connection remains unbreakable.

Part 2: Backend Middleware and Protocol Integration

The core logic of the application resides within a Node.js backend. This tier is responsible for speaking the proprietary Blackmagic protocol, exposing an API layer for web clients, and managing the local database.

2.1 The atem-connection Protocol Library

Blackmagic Design utilizes a proprietary, undocumented UDP-based networking protocol to govern switcher operations.[28] To interact with this protocol without utilizing the official C++ SDK, developers rely on the atem-connection npm package.[29] Maintained primarily by the Norwegian Broadcasting Corporation (NRK) as a critical component of their Sofie TV Automation System, this TypeScript/Node.js library is the undisputed industry standard for web-based ATEM integration.[31]

The library abstracts the complex UDP handshake, packet acknowledgment, and protocol bitmasking into a manageable JavaScript object.[30] It maintains an internal, real-time representation of the switcher's operational status, known as the AtemState object.[32] Because the library supports models ranging from legacy switchers on firmware v7.2 to the latest firmware v8.6+ [30], the middleware can remain relatively agnostic to the specific hardware deployed in the studio.

The primary operational mechanism of the library is event-driven. The software initiates a connection via myAtem.connect('IP_ADDRESS'). Once the handshake is successful, the library downloads the complete configuration of the hardware and emits a connected event.[32] From that moment forward, any change made on the physical switcher, the ATEM Software Control, or via the atem-connection library itself triggers a stateChanged(state, path) event.[32] This event broadcasts the updated AtemState object alongside a string indicating the specific JSON path that was modified. This granular path updating is critical for performance; rather than re-evaluating the entire massive state object, the Node.js application can listen exclusively for changes to routing paths (e.g., video.auxilliaries or video.mixEffects) and instantly broadcast those specific updates to the frontend UI via WebSockets.

To execute routing changes, the library provides asynchronous command methods. For example, routing Input 5 to Auxiliary Output 3 is achieved by invoking myAtem.changeAuxInput(3, 5).[32] The library handles the UDP packet generation and expects an acknowledgment from the ATEM before resolving the JavaScript Promise.

2.2 API Layer and Real-Time Synchronization

The Node.js backend must implement a robust API layer utilizing frameworks such as Express.js

or Fastify. This layer serves two primary functions: hosting the static assets of the frontend web application and exposing RESTful endpoints to process incoming routing commands from external systems like Bitfocus Companion.[34]

For real-time UI synchronization, standard HTTP polling (e.g., a web client requesting the state every 500 milliseconds) is architecturally prohibited. Polling introduces unacceptable visual latency and severely burdens the Node.js event loop with unnecessary network requests. The imperative standard is the implementation of WebSockets (via libraries such as Socket.io).[35]

When the atem-connection library fires a stateChanged event indicating a routing modification, the Node.js server intercepts this change, serializes the updated routing data, and pushes it asynchronously over the open WebSocket connection to all connected browsers. This guarantees that if an operator manually presses a cross-point button on the physical ATEM hardware [11], the Dante-style matrix grid in the web UI will visually update to reflect the new route with sub-100ms latency, maintaining perfect state synchronization across the entire broadcast ecosystem.

2.3 State Management and SQLite Database Design

A core requirement of the middleware is the ability to capture a "snapshot" of the current routing matrix and save it as a recallable preset. While saving state data as a flat JSON file is a rudimentary approach, it severely limits query efficiency and metadata expansion. SQLite is the optimal storage layer.[36] As a lightweight, serverless, file-based relational database, SQLite provides ACID compliance and highly performant read capabilities without the operational overhead of a full PostgreSQL or MySQL daemon.

When a snapshot request is triggered, the backend must traverse the active AtemState object, extract the current integer values representing the active sources on all Program, Preview, and Auxiliary buses, and map them to a relational database schema.

Table 2: Proposed SQLite Preset Database Schema

Table Name	Column	Data Type	Constraints	Description
Presets	id	INTEGER	PRIMARY KEY	Unique auto-incrementing identifier.[37]
	name	TEXT	NOT NULL	User-defined label (e.g., "Pre-Show

				Walk-In").
	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of snapshot generation.[38]
Routes	id	INTEGER	PRIMARY KEY	Unique identifier for the individual cross-point route.
	preset_id	INTEGER	FOREIGN KEY	Associates the specific route with a master preset.
	destination_type	TEXT	NOT NULL	ENUM identifier: 'AUX', 'PGM', 'PVW', 'USK'.
	destination_idx	INTEGER	NOT NULL	The specific bus index (e.g., AUX bus 4).
	source_id	INTEGER	NOT NULL	The ATEM's internal integer ID of the routed source video.

When Bitfocus Companion commands the middleware to recall "Preset 4", the Node.js server executes an inner join to retrieve all rows in the Routes table where preset_id = 4. The software then iterates through the result set, translating the rows back into sequential atem-connection commands (e.g., changeProgramInput, changePreviewInput, changeAuxInput) and firing them across the UDP bridge to the ATEM hardware.[32]

Part 3: The Dante-Paradigm Frontend User Interface

Traditional broadcast switchers rely heavily on destination-first operations: an operator selects

a destination output, such as AUX 1, and then turns a rotary dial or navigates a dropdown list to assign the source input.[4] In a dense routing environment containing up to 40 inputs and 24 outputs [3], this sequential process is fraught with friction and prone to misrouting errors during high-pressure live productions.

Audinate's Dante Controller eliminated this friction in the audio realm by introducing a two-dimensional XY grid matrix.[6] Replicating this paradigm for ATEM video switchers requires translating the logic into a high-performance web frontend utilizing React or Vue.js.[40]

3.1 UX/UI Topography and Interaction Logic

The user interface must render an expansive grid that adheres to the following topological rules:

- **X-Axis (Columns):** Represent the Destinations. These include physical auxiliary outputs, Program buses, Preview buses, and potentially Multiviewer window assignments.[8]
- **Y-Axis (Rows):** Represent the Sources. These include physical SDI/HDMI inputs, internal Media Players, Color Generators, and SuperSource outputs.[8]
- **Intersections:** Represent clickable boolean states. An active route is visually indicated by a distinctive icon or solid color block at the intersection of a source and a destination.[8]

The underlying algorithmic logic of the UI must respect the physical limitations of video routing. A single destination output (a column) can only receive one video source at any given millisecond. However, a single video source (a row) can be routed to an infinite number of destinations simultaneously.[8] Therefore, the React/Vue component must enforce mutual exclusivity along the vertical axis. When an operator clicks an empty grid intersection, the application must immediately transmit the routing command to the backend, move the active visual indicator to the newly clicked cell, and simultaneously clear the previous active indicator within that same column.

3.2 Frontend Frameworks and Grid Virtualization

Rendering a dense routing matrix in the browser presents a significant performance challenge. For an ATEM Constellation 8K, mapping 40 inputs against 24 outputs results in a grid of 960 interactive cells. If each of these cells is rendered as an independent DOM node with attached state listeners and click handlers, the React or Vue.js application will suffer from severe render lag, particularly on lower-powered tablet devices commonly used as auxiliary control surfaces.

To mitigate DOM bloat, developers must strictly avoid utilizing heavy, pre-packaged data-table libraries (such as MUI X Data Grid or traditional AG-Grid implementations).[43] These libraries are optimized for enterprise data visualization, pagination, filtering, and text editing, rather than high-speed, interactive boolean matrices.[45]

Instead, the UI should be constructed using a headless virtualization library, such as **TanStack**

Virtual (formerly React Virtual).[46] Grid virtualization ensures that the browser only renders the specific DOM nodes that are currently visible within the user's viewport. As the user scrolls across the matrix, the library recycles the DOM nodes, swapping the underlying data in real-time.[47] This architectural pattern guarantees a smooth, 60-frames-per-second scrolling experience and instantaneous click responsiveness, regardless of the overall size of the routing matrix.[46]

By pairing a virtualized grid with a global state management tool (such as Redux, Zustand, or Vuex) [49], the frontend can maintain a lightning-fast representation of the ATEM's routing status, reacting instantaneously to the WebSocket payloads pushed by the Node.js backend.

Part 4: Bitfocus Companion Bridge Integration Strategies

Bitfocus Companion is the definitive automation tool in live production, enabling operators to transform affordable USB macro keypads like the Elgato Stream Deck into highly sophisticated broadcast control surfaces.[10] The ultimate objective of the custom matrix middleware is to allow operators to build complex routing scenes in the web UI, save them to the SQLite database, and subsequently trigger them with a single physical button press on the Stream Deck.[9]

There are four primary integration pathways to bridge the custom middleware with Bitfocus Companion, each carrying distinct architectural trade-offs regarding development effort, protocol reliability, and system feedback.

4.1 Option A: REST API via Generic HTTP

The most straightforward integration strategy requires the Node.js middleware to expose standard RESTful HTTP endpoints (e.g., an Express.js route listening for POST /api/presets/recall/:id). Operators map a Stream Deck button in Companion to utilize the pre-installed "Generic HTTP" module, configuring it to fire the POST request when pressed.[34]

- **Architectural Advantages:** This approach requires minimal development overhead. HTTP over TCP guarantees packet delivery, ensuring that a button press will reliably trigger the routing change.[50] Furthermore, exposing a REST API allows the middleware to be controlled by various other third-party automation tools, scripts, or scheduling applications beyond Bitfocus Companion.[34]
- **Architectural Disadvantages:** The Generic HTTP module in Companion is highly optimized for unidirectional control.[34] Extracting complex JSON response data back into Companion to provide visual button feedback (e.g., turning a physical Stream Deck button green when "Preset 4" is currently active) is notoriously difficult. It requires complex custom variable parsing and forces Companion to continuously poll the REST API via GET requests, which introduces unnecessary performance overhead on the Companion host

machine.[51]

4.2 Option B: Open Sound Control (OSC) Listener

Alternatively, the Node.js middleware can be programmed to initialize a UDP socket listening for Open Sound Control (OSC) messages. Operators utilize Companion's "Generic OSC" module to transmit simple datagram strings, such as /matrix/preset/recall 4, when a button is pressed.[54]

- **Architectural Advantages:** OSC is a ubiquitous protocol in live production environments, designed specifically for low-latency, real-time control.[54] Crucially, Companion natively excels at receiving OSC strings.[54] The Node.js middleware can blast OSC feedback messages back to the Companion host IP address whenever a preset is activated. Companion can listen for these incoming strings and instantly change the color of the physical Stream Deck buttons to reflect the active routing state, entirely circumventing the heavy overhead of HTTP polling.[54]
- **Architectural Disadvantages:** OSC operates over UDP, which is a stateless protocol that does not guarantee packet delivery. While packet loss is rare on isolated, hardwired production subnets, a dropped UDP packet results in a missed routing cue, making it theoretically less reliable than TCP-based REST for the initial trigger command.

4.3 Option C: Custom Companion Module Development

The most deeply integrated approach requires authoring a bespoke .js module strictly for Bitfocus Companion. This module would be loaded directly into the Companion core and interface natively with the custom matrix middleware, utilizing Companion’s internal APIs to manage network connections, actions, feedbacks, and dynamic variables.[56]

- **Architectural Advantages:** This provides the ultimate "plug-and-play" experience for the end-user. A custom module can seamlessly query the middleware's SQLite database and automatically populate user-friendly dropdown menus inside Companion containing all available presets.[58] It grants direct access to Companion's native feedback engine, easily illuminating buttons based on active states without the user needing to manually configure OSC listeners.[35]
- **Architectural Disadvantages:** Bitfocus Companion transitioned to a highly decoupled, multi-process architecture starting with version 3.0.[56] Developing, compiling (via Yarn workflows), and maintaining a custom module requires strict adherence to Bitfocus coding guidelines.[57] If the core Companion application is updated, sideloaded custom modules are highly susceptible to breaking unless the developer commits to long-term, continuous maintenance.[60]

4.4 Option D: Native ATEM Macro Generation and Injection

The final option bypasses the need for the middleware to store presets internally. Instead, when an operator designs a matrix routing scene in the web UI, the Node.js application generates a

native ATEM XML Macro file.[61] Using the atem-connection library's data transfer protocols, the middleware injects this XML macro directly into the ATEM hardware's internal memory.[63] The operator then relies on Companion's existing, highly stable bmd-atem module to trigger the native macro directly on the switcher.[22]

- **Architectural Advantages:** Executing macros internally on the ATEM's FPGA hardware results in absolute zero network latency during the routing change.[61] Furthermore, this architecture removes the Node.js middleware as a critical point of failure during a live show; even if the Raspberry Pi host crashes, the routing presets live safely inside the ATEM and can be triggered via Companion or the official ATEM Software Control.[61]
- **Architectural Disadvantages:** Generating flawless ATEM XML code programmatically is highly complex.[62] ATEM switchers also possess strict internal memory limitations; an ATEM Mini is capped at storing a maximum of 100 macros [65], limiting the number of presets a user can create. Most critically, this approach exposes the system to the heavily documented "Zero-Length Macro Bug," detailed below.[22]

Recommended Integration Architecture: The optimal path balances reliability, feedback, and development overhead by combining Option A and Option B into a hybrid bridge. The Node.js middleware should strictly require TCP/HTTP POST requests from Companion to trigger the preset, guaranteeing that the execution command is successfully delivered.[34] However, upon successful execution, the middleware should transmit UDP/OSC packets back to the Companion host.[55] Companion will use these incoming OSC messages solely to update button coloring and text variables.[54] This hybrid approach secures execution reliability while providing instantaneous visual feedback, entirely circumventing the heavy maintenance burden required to author a custom Companion module.

Part 5: Edge-Case Handling and Hardware Bugs

Deploying third-party middleware into a mission-critical live broadcast environment demands rigorous error handling. Two primary edge-cases must be systematically managed to prevent catastrophic failure during a production.

5.1 Mitigating the ATEM "Zero-Length" Macro Bug

If the architectural path of utilizing native ATEM macros (Option D) is explored, developers must implement a programmatic mitigation for a documented flaw in the ATEM control protocol, universally referred to by the broadcast engineering community as the "Zero-Length Macro Bug".[22]

When an ATEM macro executes a series of routing changes (e.g., switching AUX 1 through 6 simultaneously) without any programmed delays between the commands, the ATEM processes the entire sequence in fractions of a millisecond. Bitfocus Companion's native bmd-atem module relies on polling the stateChanged data stream to detect if a macro is currently

running.[22] Because the "zero-length" macro finishes executing faster than the network polling cycle can register, Companion entirely fails to detect that the macro was triggered. Consequently, the feedback loops break, and the physical Stream Deck button fails to blink or change color to indicate that the routing command was successfully fired.[22]

Workaround Logic: If the custom middleware dynamically generates XML macros, the algorithm must programmatically inject a microscopic, artificial delay at the conclusion of the macro. By inserting the specific XML node <Op id="MacroSleep" frames="1"/> at the end of the routing sequence, the ATEM is forced to hold the macro execution state open for exactly one video frame (e.g., 16.6 milliseconds if operating at 60fps).[66] This infinitesimally brief pause is imperceptible to the human eye, but it keeps the internal state flag active long enough for the UDP network protocol to notify Companion that the macro is running, effectively resolving the feedback bug.[22]

5.2 Auto-Reconnect Logic and Invalid Requests

Because the atem-connection library relies on UDP networking, the protocol is inherently stateless. If the Ethernet connection between the middleware host machine and the ATEM is temporarily severed—due to a pulled cable or a network switch reboot—the library will drop the connection and fire a disconnected event.[30]

The Node.js backend must anticipate this behavior by encapsulating the myAtem.connect() function within a resilient retry loop utilizing exponential backoff.[32] If the connection fails, the server must continuously attempt to re-establish the handshake without requiring manual user intervention. Furthermore, if Bitfocus Companion sends an HTTP request to trigger a matrix preset while the ATEM connection is down, the REST API must not crash the Node.js process. Instead, it must catch the error and return an HTTP 503 (Service Unavailable) status code.[34] Companion can be configured to parse this specific 503 error and flash a warning color on the Stream Deck, alerting the operator to the network failure.

Part 6: High-Level Data Flow Architecture

The culmination of the hardware infrastructure, protocol library, database schema, and frontend UI establishes a highly efficient, bidirectional data flow network. The system acts as a central nervous system, ensuring that physical button presses, web UI clicks, and Stream Deck triggers all resolve into a single source of truth.

Table 3: Ecosystem Data Flow Matrix

Source System	Action / Trigger	Middleware (Node.js) Processing	Destination System	Result / Visual Feedback

		Logic		
Operator (Web UI)	Clicks grid cell (e.g., Input 1 to AUX 2).	Parses WebSocket message, executes changeAuxInput(2, 1).	**ATEM Hardware**	Physical SDI routing changes instantly.[32]
ATEM Hardware	Physical panel changes route.	Receives UDP packet, updates AtemState, fires WebSocket broadcast.[30]	**Web UI (React/Vue)**	Matrix grid visually updates via React virtual DOM state change.[33]
Operator (Web UI)	Clicks "Save Preset".	Serializes current AtemState, executes INSERT INTO Routes query.[37]	**Local Disk (SQLite)**	Preset mapping data is persisted securely to the host drive.[36]
Stream Deck	Operator presses physical button.	Companion sends HTTP POST /api/preset/4 to the Express API.[34]	**Middleware (Node.js)**	Queries SQLite, fetches the array of saved routing paths.
Middleware (Node.js)	Iterates the preset route array.	Transmits rapid, sequential UDP routing commands via atem-connection.[32]	**ATEM Hardware**	The entire matrix configuration changes to match the snapshot.
Middleware (Node.js)	Post-execution API hook.	Dispatches an OSC message (e.g.,	**Stream Deck**	Companion receives OSC, updates button

		/preset/active 4) over UDP.[54]		background color to red.[55]

Part 7: Project Readiness and Execution Roadmap

To transition this architectural concept into a deployable software product, development must be compartmentalized into sequential, testable milestones. This phased approach mitigates risk and ensures that the fundamental communication bridges are stable before complex user interfaces are constructed.

Phase 1: Protocol Discovery and Connection Singleton

Objective: Establish secure, unbreakable communication with the ATEM hardware and validate the proxy architecture.

- **Developer Task:** Initialize a Node.js project. Implement the atem-connection library. Hardcode the local IP address of the target ATEM Constellation or Mini model.[32] Write a basic script that connects, logs the current state of AUX 1 to the terminal, and successfully commands AUX 1 to change to a different input source.
- **Validation Testing:** Execute the script while monitoring the official ATEM Software Control application. The route change must reflect instantly. Disconnect the physical network cable to verify that the auto-reconnect loop functions flawlessly without crashing the script.

Phase 2: API Generation and SQLite State Management

Objective: Construct the memory layer and the translation algorithms.

- **Developer Task:** Wrap the Phase 1 script in an Express.js or Fastify server.[19] Integrate SQLite and build the Presets and Routes schema.[36] Write the translation functions that parse the complex AtemState object into relational database rows, and the reverse functions that turn database rows into atem-connection execution commands. Expose REST API endpoints for "Get Matrix", "Save Preset", and "Recall Preset".
- **Validation Testing:** Utilize a tool like Postman to send HTTP requests to the server. Create a preset, manually scramble the physical switcher routing state via the hardware panel, and use Postman to trigger the recall endpoint. The ATEM must return exactly to the saved state.

Phase 3: The Dante-Paradigm Frontend

Objective: Build the interactive XY matrix frontend grid.[6]

- **Developer Task:** Develop a React or Vue.js frontend utilizing TanStack Virtual for high-performance DOM rendering.[46] Establish WebSocket connections so the UI reacts to backend events. Bind the Y-axis to ATEM inputs and the X-axis to outputs, enforcing

mutual exclusivity logic on the columns.[8]

- **Validation Testing:** Open the web UI on an iPad or laptop. Rapidly click intersections on the virtual grid. Validate that the UI does not stutter, DOM bloat is managed, and that the physical ATEM routes follow the taps with sub-200ms latency. Press physical buttons on the ATEM and verify the web UI actively tracks the hardware changes.

Phase 4: Bitfocus Companion Bridge Integration

Objective: Map the routing logic to tactical control surfaces.

- **Developer Task:** Finalize the Express.js REST API endpoints to handle Companion HTTP requests.[34] Configure an OSC transmitter script within the Node.js backend to blast basic state strings back to the Companion host IP upon successful preset execution.[54]
- **Validation Testing:** Map an Elgato Stream Deck button via Companion's Generic HTTP module to trigger the API.[34] Map an OSC listener to the button's feedback channel.[54] Press the button; the entire ATEM matrix must reconfigure, and the physical button color must change based on the OSC feedback.[55]

Phase 5: Containerization and Hardware Deployment

Objective: Package the middleware for mission-critical production resiliency.

- **Developer Task:** Implement deep error handling to catch unhandled Promise rejections and network timeouts. Wrap the Node.js application, static frontend assets, and SQLite database into a Docker container, or compile it into a standalone executable via pkg or Electron [67], optimizing it for the target Raspberry Pi 5 architecture.[26] Configure the host OS to run the application as an auto-starting system daemon.
- **Validation Testing:** Execute a hard power pull on the Raspberry Pi host machine. Upon restoration of power, the OS must automatically launch the daemon, establish the UDP connection to the ATEM, and resume operational readiness without any human intervention.

Conclusion

The technical feasibility of engineering a Dante-style matrix routing interface for Blackmagic ATEM video switchers is exceptionally high, provided the development team strictly adheres to asynchronous, event-driven design principles.

The primary architectural trap in broadcast software development is the direct, continuous polling of hardware. By implementing a Node.js proxy server utilizing the atem-connection library [30], the middleware acts as an impenetrable shield, protecting the ATEM from the crippling concurrent connection limits that plague environments utilizing multiple direct clients.[12] By shifting the heavy computational lifting of preset storage and state-diffing to a local SQLite database and an external CPU [36], the software successfully circumvents the strict

storage limits and XML execution flaws—specifically the Zero-Length Macro Bug—inherent to native ATEM operations.[22]

Ultimately, pairing a virtualized React grid with standard REST protocols for Companion triggers [34], supplemented by OSC for visual feedback [55], yields a fault-tolerant, low-latency ecosystem. This architecture successfully modernizes the ATEM operating experience, providing technical directors with the tactical speed and visual clarity required to manage high-density routing environments like the Constellation 8K.

Works cited

1. ATEM Mini - Blackmagic Design, accessed February 21, 2026, https://www.blackmagicdesign.com/products/atemmini
2. ATEM Constellation 8K - Blackmagic Design, accessed February 21, 2026, https://www.blackmagicdesign.com/products/atemconstellation8k
3. Blackmagic Design ATEM Constellation 8K Switcher - AbelCine, accessed February 21, 2026, https://www.abelcine.com/buy/broadcasting-editing/switchers-controllers/blackmagic-design-atem-constellation-8k-switcher
4. Three Ways To Set Your Auxiliary Outputs - Black Magic Design ATEM Switcher - YouTube, accessed February 21, 2026, https://www.youtube.com/watch?v=mYn5HOOY2OM
5. ATEM Television Studio – Software | Blackmagic Design, accessed February 21, 2026, https://www.blackmagicdesign.com/products/atemtelevisionstudio/software
6. Dante Controller, accessed February 21, 2026, https://www.getdante.com/products/software-essentials/dante-controller/
7. The Ultimate Guide to the Dante Controller in Professional AV Systems - AVIXA, accessed February 21, 2026, https://www.avixa.org/pro-av-trends/articles/dante-controller-ultimate-guide
8. How do I set up audio routes on a network? - Dante, accessed February 21, 2026, https://www.getdante.com/support/faq/how-do-i-set-up-audio-routes-on-a-network/
9. Building an ATEM Switcher Control Surface with BitFocus Companion & StreamDeck, accessed February 21, 2026, https://www.youtube.com/watch?v=qT2xLEFtqk0
10. Companion - Bitfocus, accessed February 21, 2026, https://bitfocus.io/companion
11. ATEM Constellation | Blackmagic Design, accessed February 21, 2026, https://www.blackmagicdesign.com/products/atemconstellation
12. How many ethernet connections for Constellation HD 2M/E? - Blackmagic Forum, accessed February 21, 2026, https://forum.blackmagicdesign.com/viewtopic.php?f=4&t=165096
13. LEGACY - Multiple Clients Connecting to ATEM Switchers - YouTube, accessed February 21, 2026, https://www.youtube.com/watch?v=ApYouYfX5G4
14. Limited number of connections - Blackmagic Forum • View topic, accessed

February 21, 2026, https://forum.blackmagicdesign.com/viewtopic.php?f=4&t=122610
15. Multiple ATEM 1 M/E switchers get disconnected from BMD SW panels at the same time, accessed February 21, 2026, https://www.reddit.com/r/VIDEOENGINEERING/comments/ztpb26/multiple_atem_1_me_switchers_get_disconnected/
16. ATEM Network Connection Limits - Blackmagic Forum • View topic, accessed February 21, 2026, https://forum.blackmagicdesign.com/viewtopic.php?t=101841&p=564167
17. The Proxy Design Pattern in Node.js | by Mohamed-Ali - Medium, accessed February 21, 2026, https://medium.com/@moali314/the-proxy-design-pattern-in-node-js-f16f7919e3e7
18. The Proxy Pattern: A Masterpiece of Control and Illusion in Node.js - DEV Community, accessed February 21, 2026, https://dev.to/alex_aslam/the-proxy-pattern-a-masterpiece-of-control-and-illusion-in-nodejs-6dg
19. Monitoring using Proxy Design Pattern with Node.js - DEV Community, accessed February 21, 2026, https://dev.to/smhkneonix/monitoring-using-proxy-design-pattern-with-nodejs-50lb
20. OpenSwitcher - Open source control software for BMD video switchers, accessed February 21, 2026, https://openswitcher.org/
21. LEGACY - ATEM Proxy: Unlimited ATEM Switcher clients! - YouTube, accessed February 21, 2026, https://www.youtube.com/watch?v=L3GgCtEKmDA
22. Connection - bmd-atem - Bitfocus, accessed February 21, 2026, https://bitfocus.io/connections/bmd-atem
23. [BUG] Loosing connection with ATEM TVS HD after running >1h #168 - GitHub, accessed February 21, 2026, https://github.com/bitfocus/companion-module-bmd-atem/issues/168
24. connectivity issues caused by low memory? - Raspberry Pi Forums, accessed February 21, 2026, https://forums.raspberrypi.com/viewtopic.php?t=382059
25. Memory Leak? Or is it time for a bigger server? Using Raspberry Pi with 1Gb RAM - Reddit, accessed February 21, 2026, https://www.reddit.com/r/homeassistant/comments/157dxn4/memory_leak_or_is_it_time_for_a_bigger_server/
26. Raspberry Pi 4 vs Raspberry Pi 5: Which one should you choose? - Viam, accessed February 21, 2026, https://www.viam.com/post/your-ultimate-guide-to-raspberry-pi-5-now-fully-supported-on-viam
27. Raspberry Pi 5 Vs Raspberry Pi 4 Model B | Comparison & Benchmarking - Core Electronics, accessed February 21, 2026, https://core-electronics.com.au/guides/raspberry-pi-5-vs-raspberry-pi-4-model-b-comparison-and-benchmarking/
28. Welcome to the Open Switcher documentation — OpenSwitcher 0.1.0

documentation, accessed February 21, 2026, https://docs.openswitcher.org/
29. scnmtv/atem-connection: ATEM Connection library - GitHub, accessed February 21, 2026, https://github.com/scnmtv/atem-connection
30. atem-connection - NPM, accessed February 21, 2026, https://npmjs.com/package/atem-connection
31. NRK Open source, accessed February 21, 2026, https://nrkno.github.io/
32. Sofie ATEM Connection: A Part of the Sofie TV Studio Automation System - GitHub, accessed February 21, 2026, https://github.com/Sofie-Automation/sofie-atem-connection
33. atem-connection-nodered (node), accessed February 21, 2026, https://flows.nodered.org/node/atem-connection-nodered
34. generic-http - Connections - Bitfocus, accessed February 21, 2026, https://bitfocus.io/connections/generic-http
35. bitfocus/companion-module-resolume-arena - GitHub, accessed February 21, 2026, https://github.com/bitfocus/companion-module-resolume-arena
36. Memory leak..? - Raspberry Pi Forums, accessed February 21, 2026, https://forums.raspberrypi.com/viewtopic.php?t=16850
37. Schema Snapshots - DBA Dash, accessed February 21, 2026, https://dbadash.com/docs/help/schema-snapshots/
38. Database design for point in time "snapshot" of data? - Stack Overflow, accessed February 21, 2026, https://stackoverflow.com/questions/720856/database-design-for-point-in-time-snapshot-of-data
39. View topic - PGM on AUX with ATEM 1 - Blackmagic Forum, accessed February 21, 2026, https://forum.blackmagicdesign.com/viewtopic.php?f=4&t=31828
40. Rendering Mechanism - Vue.js, accessed February 21, 2026, https://vuejs.org/guide/extras/rendering-mechanism
41. Modularizing React Applications with Established UI Patterns - Martin Fowler, accessed February 21, 2026, https://martinfowler.com/articles/modularizing-react-apps.html
42. Dante Matrix Router, accessed February 21, 2026, https://adn.harmanpro.com/static/archimedia/aa_help/Workflow/Dante_Matrix_Router.htm
43. React Data Grid component - MUI X, accessed February 21, 2026, https://mui.com/x/react-data-grid/
44. Best Datagrid Library for React? : r/reactjs - Reddit, accessed February 21, 2026, https://www.reddit.com/r/reactjs/comments/1kex3t9/best_datagrid_library_for_react/
45. React Data Grids: Navigating Options for Advanced Features | by Pankaj Bhagat | Medium, accessed February 21, 2026, https://medium.com/@pankaj01/react-data-grids-navigating-options-for-advanced-features-21c2c5ea9223
46. How to build a responsive virtual grid with tanStack virtual - DEV Community, accessed February 21, 2026, https://dev.to/dango0812/building-a-responsive-virtualized-grid-with-tanstack-vi

rtual-37nn
47. Recommended pattern for a grid of dynamic, but uniformly-sized items? · TanStack virtual · Discussion #732 - GitHub, accessed February 21, 2026, https://github.com/TanStack/virtual/discussions/732
48. Best Free React DataGrids to Use in 2025 - DEV Community, accessed February 21, 2026, https://dev.to/olga_tash/best-free-react-datagrids-to-use-in-2025-2ml
49. Redux-driven UI design - by Lachlan Miller - Medium, accessed February 21, 2026, https://medium.com/@lachlanmiller_52885/redux-driven-ui-design-8586a84d808f
50. I released OpenSwitcher 0.5 - opensource ATEM Switcher control software : r/VIDEOENGINEERING - Reddit, accessed February 21, 2026, https://www.reddit.com/r/VIDEOENGINEERING/comments/ta6t81/i_released_openswitcher_05_opensource_atem/
51. Connection - studiocoast-vmix - Bitfocus, accessed February 21, 2026, https://bitfocus.io/connections/studiocoast-vmix
52. Feature Request: Add `GET` portion to new HTTP api · Issue #2808 · bitfocus/companion, accessed February 21, 2026, https://github.com/bitfocus/companion/issues/2808
53. Issues · bitfocus/companion-module-generic-http - GitHub, accessed February 21, 2026, https://github.com/bitfocus/companion-module-generic-http/issues
54. Connection - generic-osc - Bitfocus, accessed February 21, 2026, https://bitfocus.io/connections/generic-osc
55. OSC Feedback for Bitfocus Companion - da-Share, accessed February 21, 2026, https://da-share.com/forum/index.php?topic=789.0
56. Module Development Setup - Bitfocus Companion, accessed February 21, 2026, https://companion.free/for-developers/module-development/home/
57. bitfocus/companion-module-bmd-atem - GitHub, accessed February 21, 2026, https://github.com/bitfocus/companion-module-bmd-atem
58. Bitfocus' Companion module - Rundown Studio, accessed February 21, 2026, https://rundownstudio.app/docs/rundown/companion-module/
59. Companion uses 45% of CPU every 30 seconds - General - Bitfocus Community, accessed February 21, 2026, https://community.bitfocus.io/t/companion-uses-45-of-cpu-every-30-seconds/50
60. Bitfocus Companion Module - dLive Feature Suggestions, accessed February 21, 2026, https://forums.allen-heath.com/t/bitfocus-companion-module/26940
61. How are you using ATEM Macros? : r/VIDEOENGINEERING - Reddit, accessed February 21, 2026, https://www.reddit.com/r/VIDEOENGINEERING/comments/1kq211g/how_are_you_using_atem_macros/
62. ATEM Macros - Getting started, XML editing and more // Show and Tell Ep.20 - YouTube, accessed February 21, 2026, https://www.youtube.com/watch?v=WrWDecjTV4w
63. sofie-atem-connection/CHANGELOG.md at main - GitHub, accessed February 21, 2026,

https://github.com/Sofie-Automation/sofie-atem-connection/blob/main/CHANGELOG.md
64. mk-studio-code - HackMD, accessed February 21, 2026, https://hackmd.io/@ll-22-23/ryw9Mt4-i
65. Blackmagic ATEM Mini User Manual: Using Macros, accessed February 21, 2026, https://usermanual.com/support/blackmagicdesign/web-manual/atem-mini/using-macros
66. Atem/Blackmagic zoom macro help! : r/blackmagicdesign - Reddit, accessed February 21, 2026, https://www.reddit.com/r/blackmagicdesign/comments/1d6f19h/atemblackmagic_zoom_macro_help/
67. New Dante Controller : r/livesound - Reddit, accessed February 21, 2026, https://www.reddit.com/r/livesound/comments/1hlsmjx/new_dante_controller/